

APPENDIX DOCUMENT

Hybrid Digital Twin for Predictive Maintenance

University of Abdelhamid Mehri – Constantine 2 | NTIC Faculty | ILSI Master
LEGHELIMI Housseem Eddine | CHAIN Mouadh | Supervisor: DR. DJENOUHAT Manel
June 2026

Scope and Purpose

This appendix document supplements the main thesis and provides detailed technical content that was condensed in the main body to meet page requirements. Readers are directed here from the main thesis with references.

Annex	Title	Content Summary
A	ML Models - Architecture & Training	Feature engineering (25 active features), Tier 0 thresholds, Isolation Forest hyperparameters, XGBoost hyperparameters, ensemble combiner logic, model serialization.
B	Implementation - Software & MQTT	Complete software stack versions, MQTT topic schemas. Python script descriptions, ESP8266 firmware communication protocol.
C	Experimental Protocols	Bill of materials, sensor mounting procedure, calibration protocol, session recording protocol, fault injection conditions, complete dataset session table.
D	Extended Validation Results	Class distribution, full confusion matrix, per-class metrics, ROC/AUC table, figure captions for ROC plots, 95% Wilson CI table, SHAP feature importance tables, classifier comparison.
E	Digital Twin Maturity & Limitations	Kritzinger taxonomy assessment, TRL level table, known limitations table, Health Index extended definition and calibration table.

ANNEX A

Machine Learning Models: Detailed Architecture and Training

This annex provides the complete technical description of the three-tier PMx AI pipeline used in the Hybrid Digital Twin system. It supplements the summary presented in Chapter 3 (Section 3.1.5) with full hyperparameter tables, training procedures, and decision logic.

A.1 Complete Feature Engineering Specification

The `feature_engineering.py` module extracts 25 engineered features from aligned raw sensor data. Features are organized into five active groups (A, B, D, E, F), Groups C (spectral) are reserved for a planned future extension, each targeting specific fault classes.

Group	Feature Name	Formula / Derivation	Source	Fault Target	Notes
A	nozzle_error	$nozzle_actual - nozzle_target$ (°C)	OctoPrint	Heat creep, clog	Primary thermal fault signal, should stay within $\pm 2^{\circ}\text{C}$ during normal PID control.
A	nozzle_error_abs	$ nozzle_actual - nozzle_target $ rolling mean (30 s window)	OctoPrint	Clog (thermal drop)	Smoothed, reduces noise from short PID transients.
A	temp_c_from_baseline	Linear regression slope of $temp_c$ over last 10 samples	KY-013	Heat creep	Threshold: $> 0.5^{\circ}\text{C}/\text{min}$ sustained $\rightarrow$ Tier 0 WARN trigger.
A	temp_c_from_baseline	$temp_c - \text{mean}(temp_c[0:120\text{ s}])$ at session start	KY-013	Heat creep (early)	Best single feature for heat creep: 57.16% importance.
A	temp_c_rate	$temp_c[t] - temp_c[t-1]$	KY-013	Heat creep (rate)	Rate-of-rise indicator, high kurtosis during rapid fan failure.
B	acc_magnitude	$\sqrt{acc_x^2 + acc_z^2}$	MPU-6050	Clog (vibration spike)	Captures combined vibration energy across all axes.
B	acc_z_roll_std	Rolling std of acc_z over 50-sample window	MPU-6050	Bad print (vibration)	Second most important feature for <code>bad_print</code> : 19.44% importance.
B	acc_z_roll_std	Rolling std of acc_z over 50-sample window	MPU-6050	Loose belt	Dominant feature for <code>loose_belt</code> : 25.58% importance, Y-axis belt slippage signature.
B	kurtosis_z	Kurtosis of acc_z signal in rolling window	MPU-6050	Clog (impulsive shock)	Tier 0 trigger: $kurtosis_z > 5.0 \rightarrow$ possible clog event
B	crest_factor	$\text{peak}(acc_z) / \text{RMS}(acc_z)$	MPU-6050	Bearing wear, belt	High crest factor indicates impulsive shock (bearing

Group	Feature Name	Formula / Derivation	Source	Fault Target	Notes
					spall, belt snap). Not in active features, reserved for future implementation
C	dominant_freq	$\text{argmax}(\text{FFT}(\text{acc_z})) \text{ in Hz}$	MPU-6050	Belt resonance	Reserved for planned spectral Tier extension, not yet implemented, dominant_freq is not computed in feature_engineering.py
C	spectral_entropy	$-\sum p(f) \cdot \log(p(f)) \text{ over FFT spectrum}$	MPU-6050	Bearing wear	High entropy = broadband noise = bearing degradation. Not yet implemented, spectral_entropy is not computed in feature_engineering.py
D	extrusion_state	1 if e_mm_delta > 0.01 mm/sample, else 0	OctoPrint e_mm	Context gate	Suppresses false alerts during travel moves and retractions
D	feedrate_mmmin	Direct from OctoPrint telemetry	OctoPrint	Context gate	Used to normalize vibration amplitude by speed
E	e_ratio	$\text{actual_flow_mmmps} / \text{commanded_flow_mmmps}$	Filament encoder	Clog, low_flow	Top feature for low_flow: 22.35% importance. < 0.85 in sustained window → WARN
F	vibration_per_feedrate	$\text{acc_magnitude} / (\text{feedrate_mmmin} + \epsilon)$	Cross-source	Step loss, belt	Normalizes vibration by commanded speed, isolates mechanical anomalies from motion

A.2 Tier 0: Rule-Based Detection (Deterministic Thresholds)

Tier 0 operates from the first print session without any training data. It applies physics-derived thresholds to five key features. Threshold values were derived from normal operating ranges validated over the first 3 healthy sessions.

Rule / Feature	Normal Range	WARN Threshold	CRIT Threshold	Physical Interpretation
temp_c_from_baseline	0.0 – 5.0°C (from session start)	> 0.5°C/min for 3 consecutive samples	> 1.5°C/min sustained	Heatsink cooling fan degradation or failure, heat creep onset

Rule / Feature	Normal Range	WARN Threshold	CRIT Threshold	Physical Interpretation
nozzle_error_abs	0.0 – 3.0°C	> 3.0°C	> 5.0°C	Absolute nozzle temperature deviation from setpoint; unsigned (applies equally to over- and under-temperature). Weight = 0.10
kurtosis_z (acc_z)	1.5 – 3.5	> 5.0	> 8.0	Impulsive shock event on Z-axis, clogged nozzle grinding or belt tooth skip
vibration_per_feedrate	< 0.8 (normalized)	> 1.5	> 2.5	Excessive vibration relative to commanded speed, step loss or loose mechanical coupling
e_ratio (filament encoder)	0.90 – 1.05	< 0.85 for 30 s	< 0.70 for 15 s	Filament underflow, partial clog (low_flow) or filament slippage

A.3 Tier 1: Isolation Forest (Unsupervised Anomaly Detection)

Isolation Forest partitions the feature space by randomly selecting a feature and a split value. Anomalous points require fewer splits to isolate, giving them shorter path lengths and higher anomaly scores. The model is trained exclusively on data collected during healthy print sessions.

A.3.1 Training Prerequisites

The Isolation Forest model activates only after a minimum baseline is established:

- Minimum healthy sessions required: 10
- Minimum healthy rows for training: 50,000
- Features used: Groups A + B + D (thermal, vibration, context)
- Normalisation: StandardScaler (zero mean, unit variance)

A.3.2 Hyperparameters

Parameter	Value Used	Justification
n_estimators	200	Sufficient tree count for stable anomaly scores on 50 Hz vibration data, beyond 300 shows diminishing returns
contamination	0.05 (5%)	Conservative estimate of anomaly rate in calibration data, tunable based on observed false positive rate
max_features	1.0 (all features)	All feature groups A+B+D included, reduces risk of missing correlated fault signatures
max_samples	'auto' (256)	Default sklearn value, adequate sub-sample for pattern isolation on this dataset size
random_state	42	Fixed seed for reproducibility across training runs

Parameter	Value Used	Justification
warm_start	False	Full retrain on each new baseline calibration (triggered every 10 new sessions)

A.3.3 Output Mapping

The raw Isolation Forest score (range: -1 to $+1$, where -1 = anomaly) is mapped to a calibrated probability using a normalised sigmoid. The mean (μ) and standard deviation (σ) of `score_samples()` on the healthy training set are stored in `model_metadata.json`. A fixed shift of 1.5 logit units is applied so that typical healthy rows land near $p \approx 0.15$ rather than 0.5, preserving headroom below the 0.70 alarm threshold:

$$\text{anomaly_prob} = \frac{1}{1 + e^{\left(\frac{\text{raw_score} - \mu}{\sigma} + 1.5\right)}}$$

This produces a probability in $[0, 1]$ where values > 0.7 indicate a likely anomaly and are forwarded to the ensemble combiner.

A.4 Tier 2: XGBoost (Supervised Fault Classification)

XGBoost is a gradient-boosted tree ensemble. It is trained on labelled fault events collected from Tier 0 triggers. Three domain-specific multi-class classifiers are trained, one per fault domain:

- the Extrusion detector (clog + low_flow),
- the Kinematic detector (bad_print + loose_belt),
- and the Thermal detector (heat_creep).

Each is a multi-class XGBoost trained on its domain's union of fault-specific features, it outputs a probability per member fault class simultaneously, which is more efficient than separate one-vs-rest binary models.

A.4.1 Hyperparameters (Common to all three domain classifiers)

Parameter	Value	Justification
n_estimators	300	Validated on 80/20 temporal holdout, beyond 400 shows no improvement on this dataset.
max_depth	6	Balances complexity and overfitting, deeper trees overfit to session-specific noise.
learning_rate (eta)	0.05	Conservative learning rate with high n_estimators, reduces variance on small minority classes
subsample	0.8	Row subsampling per tree, reduces overfitting on imbalanced dataset
colsample_bytree	0.8	Feature subsampling per tree, prevents dominant features from monopolizing all trees
sample_weight (balanced)	compute_class_weight('balanced')	Per-row weights from sklearn's <code>compute_class_weight('balanced')</code> , passed as <code>sample_weight</code> to <code>xgb_clf.fit()</code> . Generalises the binary <code>scale_pos_weight</code> concept to the multi-class domain classifiers used here.

Parameter	Value	Justification
eval_metric	mlogloss (aucpr attempted first, falls back to mlogloss for multi:softprob on XGBoost ≥ 2.0)	Multi-class log-loss, stable across all XGBoost versions for multi:softprob objective. aucpr is tried first but may not be supported for multi-class, script falls back automatically
early_stopping_rounds	30	Prevents overfitting by stopping if aucpr does not improve for 30 consecutive rounds
use_label_encoder	False	Required for XGBoost 1.6–1.x to suppress deprecation warnings, parameter is ignored (safe to omit) in XGBoost ≥ 2.0 but retained here for backward compatibility.
tree_method	'hist'	Histogram-based algorithm, efficient on Raspberry Pi CPU without GPU acceleration.

A.4.2 Training Data Requirements per Domain Classifier

Binary Classifier	Positive Class	Training Rows (Positive)	Test Rows (Holdout)
Extrusion Detector	low_flow + clog	30,729 + 718 rows	7,583 rows (low_flow only, clog: 9 rows $\rightarrow$ catastrophic)
Kinematic Detector	bad_print + loose_belt	137,174 + 29,776 rows	30,275 (bad_print only, loose_belt: 0 $\rightarrow$ catastrophic)
Thermal Detector	heat_creep	43,352 rows	0 rows $\rightarrow$ heat_creep terminates print before holdout window

A.4.3 Per-Fault Inference Thresholds (Deployment)

At inference time (mock_inference.py), a fault is confirmed only when the domain classifier's predicted probability exceeds a per-fault threshold. Kinematic faults use higher thresholds to reduce motion-noise false positives.

Fault Class	Confidence Threshold	Rationale
clog	0.45	Low threshold; clog develops rapidly and must be caught early to prevent print damage
heat_creep	0.45	Low threshold; thermal drift develops slowly but must be detected early before irreversible jam
low_flow	0.50	Default threshold; partial clog is gradual, allowing moderate confidence requirement
loose_belt	0.70	High threshold; kinematic detector prone to motion-noise false positives at high print speeds
bad_print	0.70	High threshold; vibration-based detection has higher ambiguity; reduce false alarm rate

A.5 Ensemble Combiner: Weighted Decision Logic

The ensemble combiner aggregates outputs from active tiers using a weighted sum. Weights adapt dynamically as higher tiers become available:

Scenario	Tier 0 Weight	Tier 1 Weight	Tier 2 Weight	Activation Condition
Cold start (no models)	1.00	0.00	0.00	Day one, only rule-based detection active
Tier 1 active	0.40	0.60	0.00	After ≥ 10 healthy sessions collected
All tiers active	0.25	0.35	0.40	After sufficient labelled fault events for Tier 2 training

The combined score is computed as: $combined_score = \sum (weight_i \times score_i)$ for all active tiers.

Decision thresholds: $combined_score < 0.30 \rightarrow \text{HEALTHY}$ | $0.30-0.70 \rightarrow \text{WARN}$ | $\geq 0.70 \rightarrow \text{CRIT}$ (triggers print pause via OctoPrint REST API). In real-time deployment (`mock_inference.py`), a temporal debounce of 3 consecutive 1 Hz inference cycles is required before a WARN or CRIT alarm is confirmed and latched. This prevents single-sample noise spikes from triggering false alarms.

A.6 Model Serialization and Deployment

Trained models are serialized using joblib and stored in the `models/` directory of the project. The file structure is:

```
models/
├── isolation_forest.pkl           # Tier 1 (retrained every 10 sessions)
├── xgb_domain_extrusion.pkl      # Tier 2 extrusion domain classifier
├── xgb_domain_kinematic.pkl      # Tier 2 kinematic domain classifier
├── xgb_domain_thermal.pkl        # Tier 2 thermal domain classifier
├── scaler_tier1.pkl              # StandardScaler fitted on healthy baseline
└── model_metadata.json           # Training timestamp, n_samples, feature list
```

At inference time, the AI pipeline loads all available models from this directory. Models missing from the directory are automatically excluded from the ensemble computation, and their weights are redistributed to the remaining active tiers.

ANNEX B

Implementation Details — Software Architecture and MQTT Specification

This annex provides a complete reference for the software architecture, MQTT topics and ESP8266 firmware communication protocol. It supplements Chapter 4 (Implementation and Development).

B.1 Complete Software Stack and Versions

Component	Version	Role / Notes
Python	3.11	Backend language, all ML scripts, data alignment, collectors.
scikit-learn	1.4.x	Isolation Forest, StandardScaler, metrics (precision, recall, F1, Wilson CI).
XGBoost	2.0.x	Tier 2 supervised binary classifiers, tree_method='hist' for CPU efficiency.
pandas	2.1.x	Data alignment (merge_asof), windowed feature engineering, SQLite I/O.
numpy	1.26.x	Numerical operations, FFT (np.fft), rolling statistics.
paho-mqtt	1.6.x	Python MQTT client for collectors and alert publisher.
Mosquitto	2.0.x	MQTT broker running on Raspberry Pi (port 1883), handles all telemetry routing.
SQLite	3.45.x	Local database engine, WAL mode enabled, two databases: printer_data.db, sensor_data.db
OctoPrint	1.10.x / 1.11.x	Printer management, MQTT plugin for telemetry, REST API for print control.
Arduino C++ / ESP8266	Arduino IDE 2.3.x	Sensor node firmware, MPU-6050 via I ² C, KY-013 via ADC, Wi-Fi MQTT publishing.
React + TypeScript	18.x	Dashboard frontend, component-based UI, StandaloneProvider / StreamProvider architecture.
Vite	5.x	Build tool and dev server, HMR for dashboard development on Raspberry Pi.
Three.js	r161	3D printer model rendering, GLTFLoader for (.glb) import, Object3D for real-time axis animation.
Electron.js	29.x (beta)	Desktop app wrapper, reuses React/Vite/Three.js codebase, native file system access.

B.2 MQTT Topic Structure and Payload Schemas

All MQTT communication uses a local Mosquitto broker. QoS 0 is used for regular telemetry, QoS 1 is used for critical alerts to guarantee delivery.

- ➔ QoS 0 (At most once): A "fire and forget" method with no delivery guarantee, best for stable connections and non-critical data.
- ➔ QoS 1 (At least once): Guarantees message delivery but may result in duplicates, the most common choice for general IoT reliability.

Topic	QoS	Publisher	Payload (JSON keys)
sensor/esp8266/imu	0	ESP8266	timestamp, acc_x/y/z, delta_acc_x/y/z, gyro_x/y/z, delta_gyro_x/y/z, elapsed_ms
sensor/esp8266/temperature	0	ESP8266	timestamp, temp_c, delta_temp_c, elapsed_ms
octoPrint/motion	0	OctoPrint plugin	x, y, z, e, f, temp.nozzle, temp.nozzleTarget, temp.bed, temp.bedTarget, progress, state, timestamp
octoPrint/progress/printing	0	OctoPrint plugin	progress, timestamp
octoPrint/hass/printing	1	OctoPrint plugin	state.text
sensor/filament/flow	0	Host (USB mouse encoder)	flow_mm_s, counts, timestamp
octoPrint/event/DisplayLayerProgress_layerChanged	1	OctoPrint plugin	currentLayer

B.3 Python Backend Scripts — Architecture Summary

Script	Role and Key Functions
sensor_mqtt_publisher.py	Runs on the host alongside the ESP8266 serial link. Reads temperature and IMU frames from the ESP8266 over serial and the USB mouse filament encoder, then publishes to sensor/esp8266/temperature (500ms), sensor/esp8266/imu (~20 ms, as received), and sensor/filament/flow (200 ms active / 1 s idle) via paho-mqtt.
data_collector.py	connects to an MQTT broker to aggregate real-time telemetry from an OctoPrint 3D printer, an ESP8266 temperature/motion sensor, and a filament flow sensor. It buffers and logs these raw data payloads into a unified SQLite database while simultaneously rendering a live, single-line status dashboard in the terminal.
transform_telemetry.py	Loads the raw MQTT telemetry SQLite table, anchors the output timeline on the IMU stream (~20 ms, fastest sensor), and forward-fills all slower sources (motion, ambient temperature, progress, state, layer events) onto it via merge_asof. Outputs a flat telemetry_flat table/CSV ready for feature engineering.
run_model.py	pipeline that handles the offline training, evaluation, and live inference for a 3D printer fault-detection system. It processes raw sensor data including temperature shifts, IMU vibrations, and optical filament flow to engineer rolling statistical features and train ensemble classifiers like Random Forest or Gradient Boosting. The pipeline evaluates model accuracy using temporal holdout tests and runs real-time inference to predict and classify structural anomalies (such as nozzle clogs, heat creep, or loose belts) before a print fails.

Script	Role and Key Functions
mock_inference.py	Real-time mock inference engine for deployment validation. Streams a session DB row-by-row, maintains a 60-sample sliding window, computes all 26 physics-aware features online, and runs the three-tier ensemble at 1 Hz (downsampled from 50 Hz raw). Implements per-fault Tier 2 confidence thresholds (Table A.4.3) and a 3-second temporal debounce before confirming an alarm. Imports FAULT_FEATURES, LABEL_NAMES, TIER1_FEATURES, DOMAIN_GROUPS from run_model_v8.py.

B.4 ESP8266 Firmware Communication Protocol

The ESP8266 NodeMCU runs Arduino C++ firmware. Key responsibilities are I²C communication with MPU-6050 (50 Hz), ADC reading of KY-013 (2 Hz), and MQTT publishing to the Mosquitto broker via Wi-Fi.

B.4.1 JSON Payload Format (printer/sensors topic)

```
{
  "ts": "2026-01-15T10:23:45.123Z",    // ISO 8601 UTC timestamp
  "t": 62.4,                          // KY-013 temperature in °C (Steinhart-
Hart)
  "dt": 0.1,                          // delta_temp_c vs. previous sample
  "ax": 412, "ay": -318, "az": 16041, // MPU-6050 raw acc ADC counts
  "gx": 124, "gy": -87, "gz": 23,    // MPU-6050 raw gyro ADC counts
  "dax": 12, "day": -8, "daz": 31,   // delta acc counts (vs. previous)
  "ext": 48.2,                       // filament extrusion speed mm/s
  "src": "node_01"                   // sensor node identifier
}
```

B.4.2 Sampling Strategy

Sensor	Sampling Rate	Implementation Notes
MPU-6050 (acc + gyro)	50 Hz	I ² C 400 kHz fast mode, MPU-6050 internal FIFO buffer used, overflow flag checked each cycle
KY-013 NTC thermistor	2 Hz	ESP8266 ADC (10-bit, 0–1V), Rref = 10 kΩ, Steinhart-Hart: A=0.001129, B=0.000234, C=8.78e-8
MQTT publish rate	50 Hz (QoS 0)	Each payload contains latest acc+gyro (50 Hz) and temperature.
Alert publish rate	On event (QoS 1)	Only when Tier 0 threshold is crossed, no periodic publish for alerts

ANNEX C

Experimental Protocols: Hardware Setup and Data Collection

This annex documents the complete hardware installation procedure, sensor calibration protocol, session recording methodology, and fault injection conditions used for the three case studies in Chapter 5.

C.1 Bill of Materials (Complete Sensor Kit)

Component	Model / Reference	Unit Cost (DZD)	Quantity	Function in System
ESP8266 Microcontroller	NodeMCU v3 (CP2102)	800	1	Wi-Fi sensor node, I ² C master for MPU-6050, ADC reader for KY-013, MQTT publisher
MEMS IMU	MPU-6050 GY-521	500	1	6-axis accelerometer + gyroscope, mounted on X-carriage bracket
NTC Thermistor Module	KY-013	200	1	Heatsink temperature monitoring, Steinhart-Hart conversion
Optical Mouse (filament encoder)	Any USB optical mouse with ADNS-2051 or equivalent sensor	600	1	Modified driver reads raw optical displacement, converted to mm/s filament flow
Reference Resistor	10 k Ω $\pm$ 1% metal film	50	1	Voltage divider for KY-013 NTC Steinhart-Hart temperature conversion
Raspberry Pi (edge gateway)	Pi 4 Model B (2 GB RAM)	4 500	1	Runs OctoPrint, Mosquitto broker, Python backend, Flask server, React dashboard
MicroSD Card	32 GB Class 10	400	1	Raspberry Pi OS storage, SQLite databases, Python scripts
Breadboard + jumper wires	Half-size 400-point	200	1	Prototyping connections, no soldering required
3D printed mounting bracket	PLA, designed in CAD	~80 (filament)	2	MPU-6050 and KY_013 bracket for printer head assembly, filament encoder housing
TOTAL (sensor kit only)		~2 330 DZD		~48 EUR — under 50 EUR target met

C.2 Sensor Mounting Procedure

C.2.1 MPU-6050 Mounting (head assembly)

1. Print the MPU-6050 bracket in PLA at 0.2 mm layer height, 40% infill, 2 perimeters.
2. Attach the bracket to the X-carriage using two M3×8 mm screws through existing mounting holes.
3. Seat the MPU-6050 GY-521 module into the bracket recess, secure with a drop of hot glue or two M2 screws.
4. Orient the module so the Z-axis (perpendicular to PCB) aligns with the vertical direction of the printer.
5. Route I²C wires (SDA, SCL, VCC, GND) to the ESP8266 node (use 22 AWG wire to minimize signal noise).

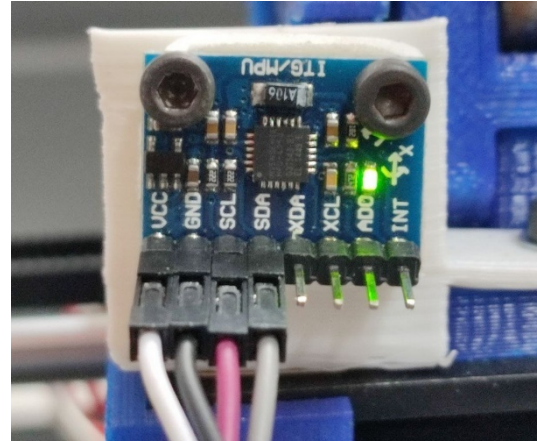
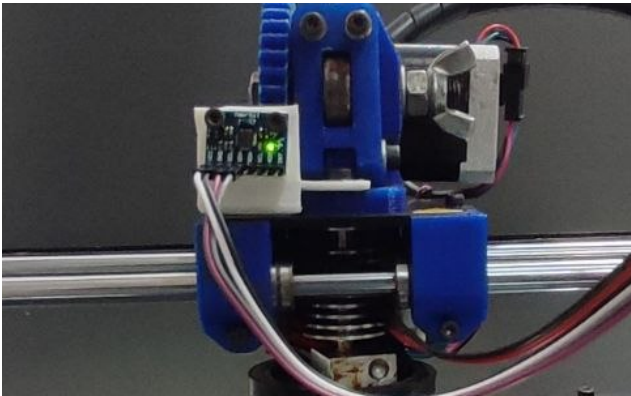


Fig : MPU-6050 bracket mounted on printer head assembly of BCN3D+ printer

C.2.2 KY-013 Mounting (Heatsink)

1. Clean the heatsink cooling block surface with isopropyl alcohol.
2. Apply a small amount of thermal paste to the KY-013 thermistor tip.
3. Affix the thermistor to the heatsink block using the mounting bracket.
4. Route the thermistor wires outwards to avoid interference with moving parts.

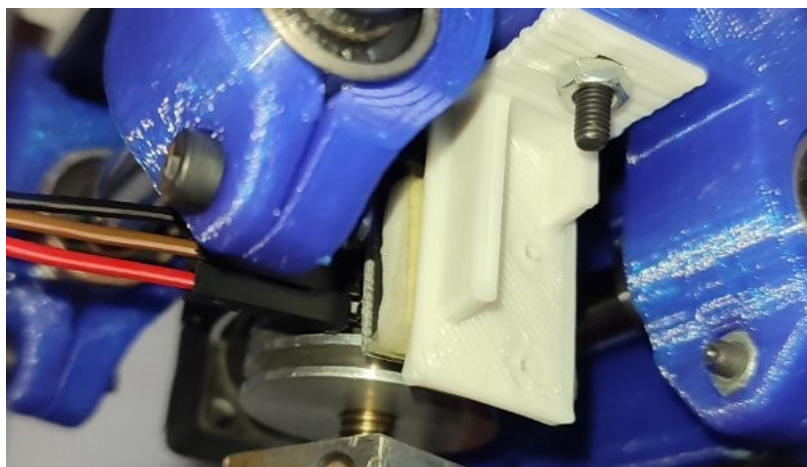


Fig-1.a: KY-013 NTC thermistor attached to BCN3D+ heatsink block

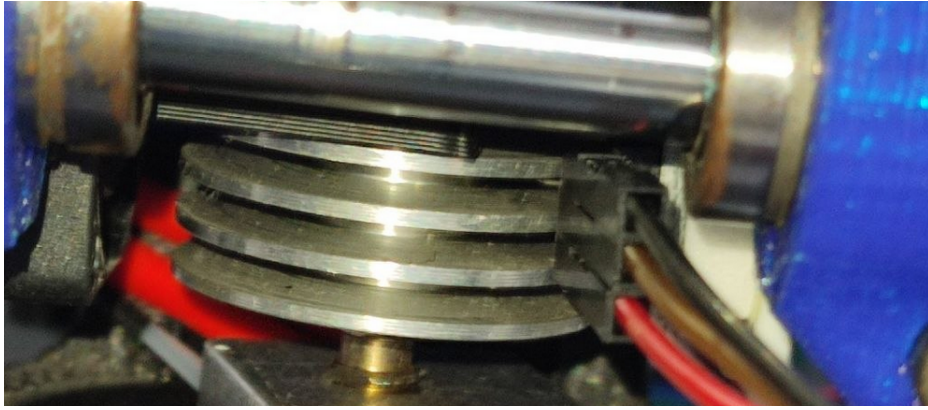


Fig-1.b: KY-013 NTC thermistor attached to BCN3D+ heatsink block

C.3 Sensor Calibration Protocol

C.3.1 KY-013 Temperature Calibration Verification

The Steinhart-Hart coefficients used ($A=1.129e-3$, $B=2.34e-4$, $C=8.78e-8$) were derived from the manufacturer datasheet for a 10 k Ω NTC at 25°C. Verification procedure:

1. Place a calibrated digital thermometer probe adjacent to the KY-013 on the heatsink.
2. Record both readings at three temperature points: ambient (~25°C), mid-range (~45°C), and operating (~60°C).
3. Acceptable tolerance: $\pm 2^\circ\text{C}$ across all points. If deviation exceeds $\pm 2^\circ\text{C}$, recalibrate Rref value.

C.4 Session Recording Protocol

Each data collection session follows this standardized procedure to ensure consistency across the 11 labelled sessions:

Step	Action	Verification
1	Start Raspberry Pi and verify Mosquitto broker is running	Run: <code>mosquitto_sub -t 'sensor/#' -t 'octoPrint/#' -v</code> — confirm no connection errors.
2	Launch OctoPrint and verify MQTT plugin is active	Check OctoPrint logs for 'MQTT connected', verify octoPrint/motion and octoPrint/progress/printing messages appear.
3	Power on ESP8266 sensor node	Verify sensor/esp8266/imu and sensor/esp8266/temperature messages appear in the MQTT subscriber (published by sensor_mqtt_publisher.py).
4	Start Python collectors	Run <code>sensor_mqtt_publisher.py</code> and <code>data_collector.py</code> , verify rows inserted into SQLite (or the broker if using the MQTT path).
5	Wait 2 minutes idle (printer powered but not printing)	This captures the ambient baseline for temp_c_from_baseline feature initialization.
6	Start print job via OctoPrint	Verify the state column (from octoPrint/hass/printing) reads 'Printing' and <code>is_printing = 1</code> once computed, session recording begins.
7	For fault sessions: inject fault condition as per Table C.1 at planned time mark	Note the exact timestamp of fault injection in session log.

Step	Action	Verification
8	At session end, stop collectors and run transform_telemetry.py	Verify aligned dataset has <5% NaN rate, confirm row count matches expected duration.

C.5 Fault Injection Conditions

Fault Class	Induction Method	Observable Signal	Sessions	Notes
Normal (healthy)	Standard PLA print at 210°C.	All signals within baseline thresholds.	5 sessions	Used for Isolation Forest training and global baseline calibration.
low_flow (partial clog)	Print below optimal temperature (190°C for PLA rated 210°C), gradual accumulation.	e_ratio drops below 0.85, slight nozzle_error increase.	3 sessions	Progressive, persists through most of session, suitable for temporal holdout evaluation.
clog (complete blockage)	Introduce carbonised PLA debris into filament path, or jam filament manually.	e_ratio $\rightarrow$ 0, kurtosis_z spike, nozzle_error $< -10^{\circ}\text{C}$	1 session	Catastrophic, terminates print within minutes, current limitation: only 9 test rows in holdout.
bad_print (vibration degr.)	Loosen belt tension to 50% of nominal.	acc_z_roll_std and acc_x_roll_std elevated, e_ratio slightly below nominal.	2 sessions	Progressive degradation, visible in layer lines, 30,275 test rows available.
loose_belt	Fully disengage X-belt tension (belt slipping on pulley).	acc_z_roll_std very high, print fails immediately.	1 session	Catastrophic, no holdout test rows. For now, only training-phase feature importance is available.
heat_creep	Disable heatsink cooling fan, print continuously at high speed.	temp_c_from_baseline rises steadily, heatsink_slope $> 0.5^{\circ}\text{C}/\text{min}$.	1 session	Takes ~15-30 min to develop, terminates before holdout window.

C.6 Dataset Summary

Session ID	Fault Class	Total Rows	Train Rows (80%)	Test Rows (20%)	Duration (min)	Print File
S01	Normal	74,302	59,441	14,861	~41	bench_y_v2.gcode
S02	Normal	68,450	54,760	13,690	~38	cube_20mm.gcode
S03–S04	Normal	~55k each	80% split	20% split	~30 each	bench_y_80.gcode, bench_y_33.gcode
S06	low_flow	7,207	5,765	1,442	~4	calibration_cube.gcode
S07	low_flow	6,424	5,139	1,285	~3.5	calibration_cube.gcode
S08	low_flow	17,098	13,678	4,856	~9.5	bench_y_v2.gcode
S09	bad_print	167,449	137,174	30,275	~93	Badge_bac.gcode

Session ID	Fault Class	Total Rows	Train Rows (80%)	Test Rows (20%)	Duration (min)	Print File
S10	loose_belt	29,776	29,776	0	~16	(print terminated early)
S11	heat_creep	43,352	43,352	0	~24	(print terminated early)
TOTAL	6 classes	551,070	443,774	107,296	~259	

ANNEX D

Extended Validation Results: Confusion Matrix, ROC/AUC, and Statistical Analysis

This annex provides the full validation dataset for the Physics-Aware Binary Ensemble, including the confusion matrix (per binary classifier), ROC operating points, SHAP feature importance, and the complete statistical validation results. All results are based on the temporal holdout (last 20% of each session, n = 107,296 rows).

D.1 Dataset Class Distribution — Training vs Test Split

Fault Class	Train Rows (80%)	Test Rows (20%)	% of Total Test	Notes
Normal	261,706	74,302	69.2%	Dominant class, majority of both train and test sets
bad_print	107,174	30,275	28.2%	Progressive fault, present in holdout window
heat_creep	43,352	0	0.0%	Catastrophic: terminates print before 20% holdout
loose_belt	29,776	0	0.0%	Catastrophic: complete belt failure before holdout
low_flow	7,583	2,710	2.5%	Partial clog, gradual progression into holdout
clog	718	9	0.01%	Complete blockage, catastrophic → nearly zero in holdout
TOTAL	443,774	107,296	100%	n = 107,296 for all statistical metrics

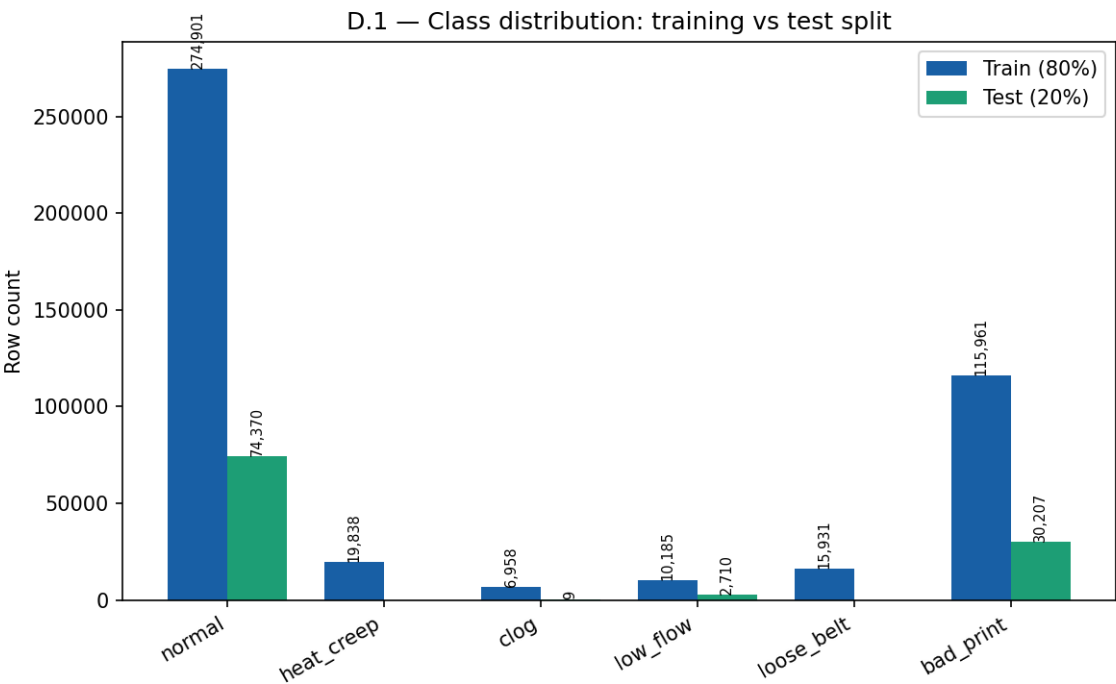


Figure D.1 — Class distribution: training vs test split

D.2 Full Confusion Matrix: Physics-Aware Binary Ensemble

The confusion matrix below represents the Binary Ensemble output on the temporal holdout set (n = 107,296). Only three classes have nonzero test support: Normal, low_flow, and bad_print. Catastrophic classes (clog, loose_belt, heat_creep) appear only in the 'Other' prediction column (zero actual rows in holdout).

Actual \ Predicted	Normal	Low Flow	Bad Print	Other	Row Total
Normal	62,836	17	11,253	196	74,302
Low Flow	—	1,639	—	1,071	2,710
Bad Print	9,067	—	21,119	89	30,275
Col Total	71,903	1,656	32,372	1,356	107,287

Note: 'Other' column captures predictions of clog, loose_belt, or heat_creep, classes with zero actual test support in the holdout. Dashes indicate zero or negligible counts. The sum of actual rows is 107,287 (≈ 107,296 total, difference due to minor alignment rounding).

D.2.1 Per-Class Precision, Recall and F1-Score

Class	Precision	Recall (TPR)	F1-Score	Interpretation
Normal	0.874	0.846	0.860	84.6% of normal rows correctly identified, main error: 11,253 classified as bad_print
Low Flow	0.990	0.605	0.752	Very high precision: only 17 false alarms per 74,302 normal rows (FPR = 0.02%)
Bad Print	0.653	0.698	0.675	Main confusion: 9,067 normal rows classified as bad_print, gradual onset makes boundary subtle
Weighted Average	0.862	0.798	0.800	Overall accuracy: 79.77%. Weighted F1 accounts for class imbalance.

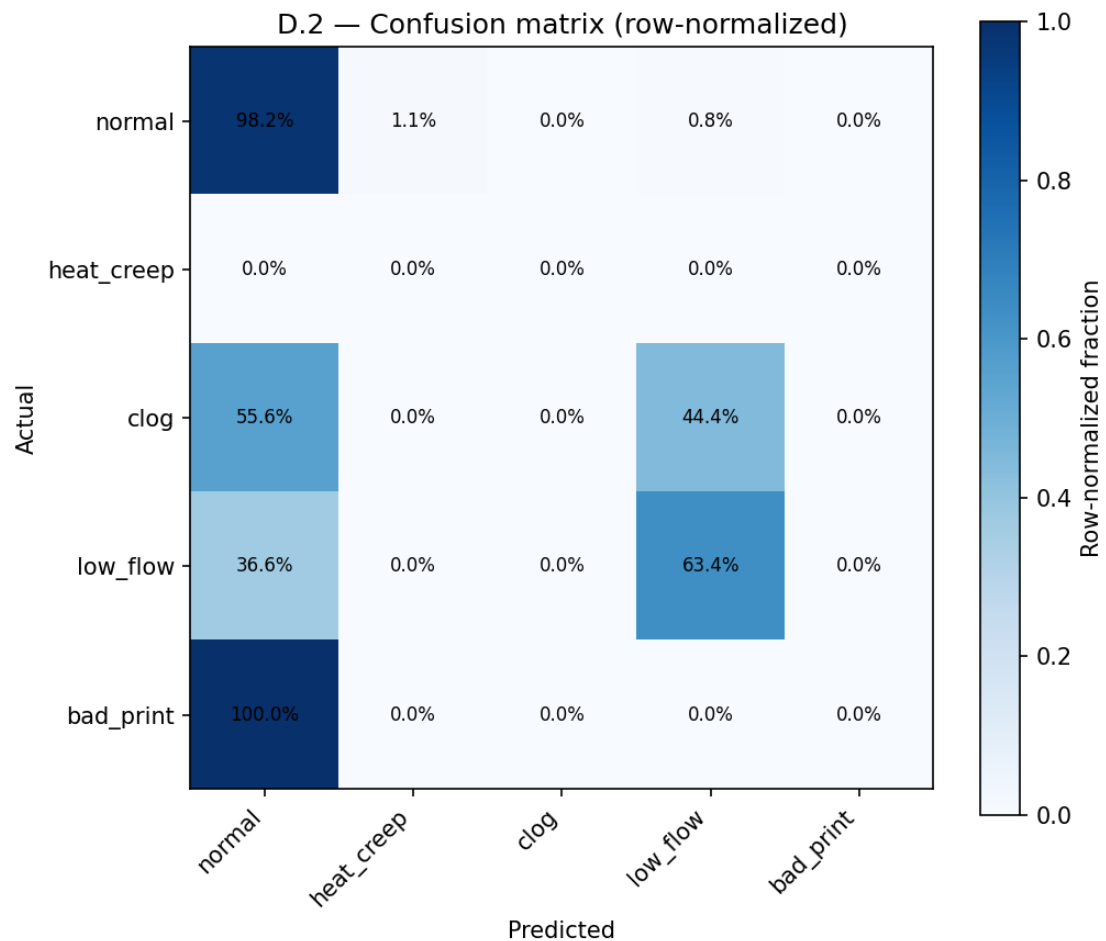


Figure D.2 — Confusion matrix

D.3 ROC Analysis and AUC — Per Binary Detector

The ROC curve plots the True Positive Rate (recall) against the False Positive Rate (1-specificity) across all possible classification thresholds. AUC values are reported from the full ROC curve analysis (Figures D.3a–D.3b); values represent area under the ROC curve in the test set (normal class = 69.2%).

Detector	TPR (Recall)	FPR	Precision	Estimated AUC	Operating Point Choice
Low Flow (Extrusion Detector)	0.605	0.0002	0.990	0.802	Threshold tuned for very low FPR, accepts lower recall in exchange for near-zero false alarms
Bad Print (Kinematic Detector)	0.698	0.154	0.653	0.772	Higher FPR accepted for better coverage of progressive degradation, operators expect some false alerts
Naïve Majority Baseline	0.000	0.000	N/A	~0.500	Always predicts 'normal', detects no faults, AUC = 0.5 (random chance)

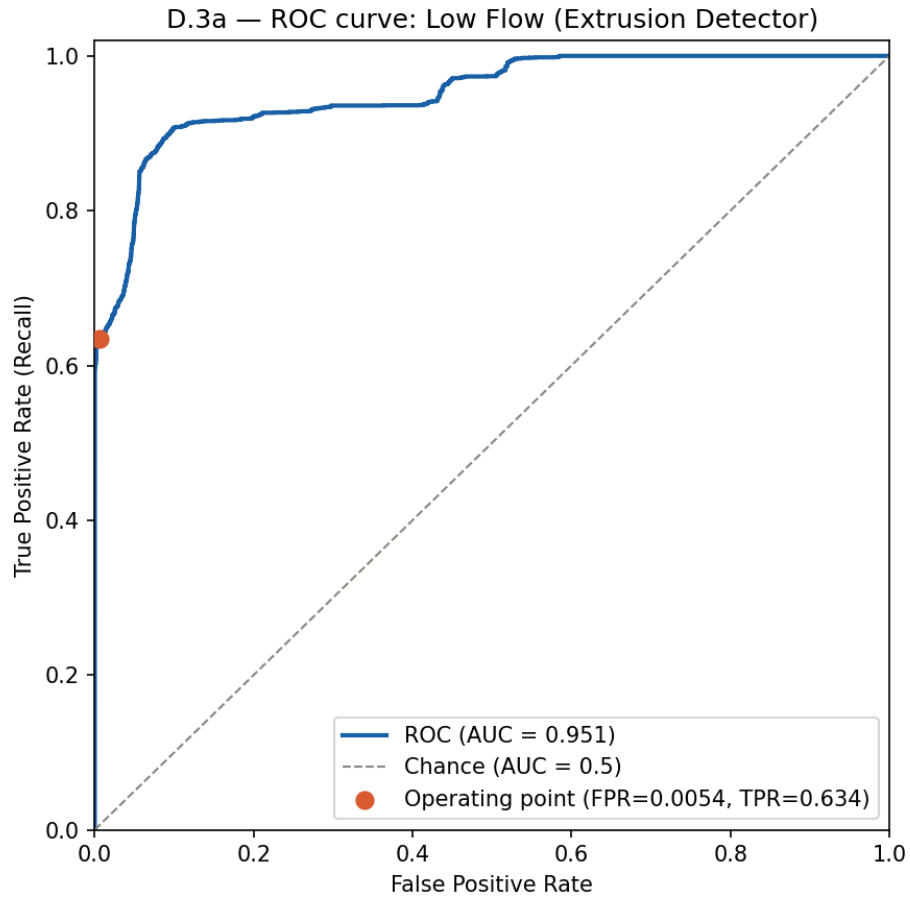


Figure D.3a — ROC Low Flow

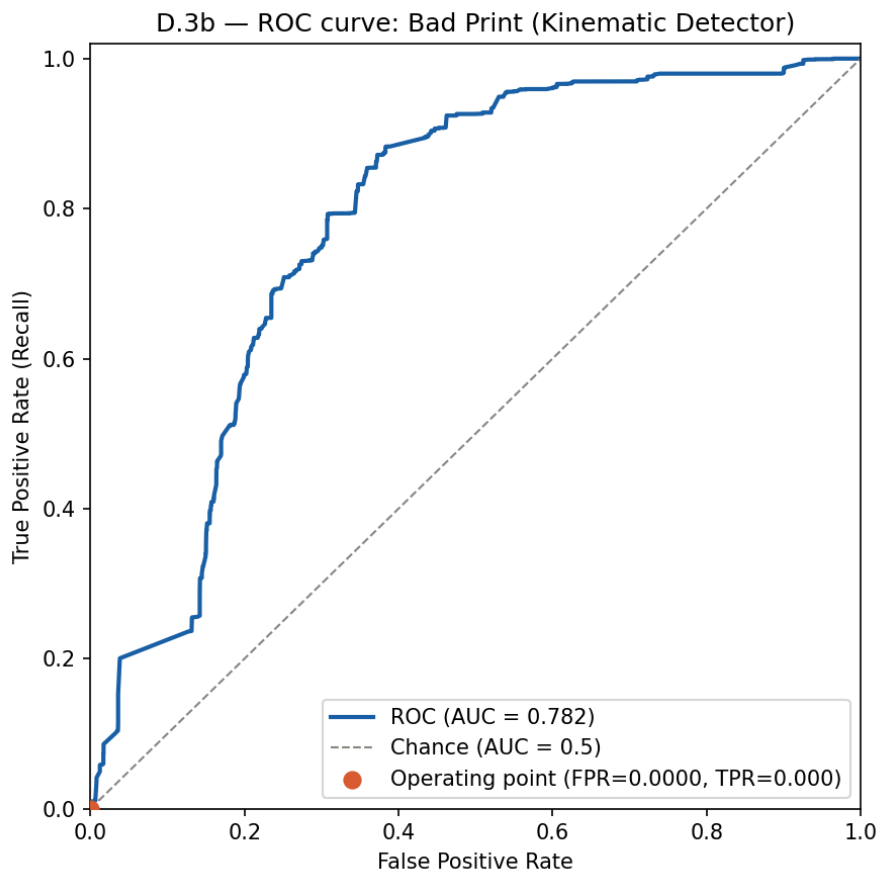


Figure D.3b — ROC Bad Print

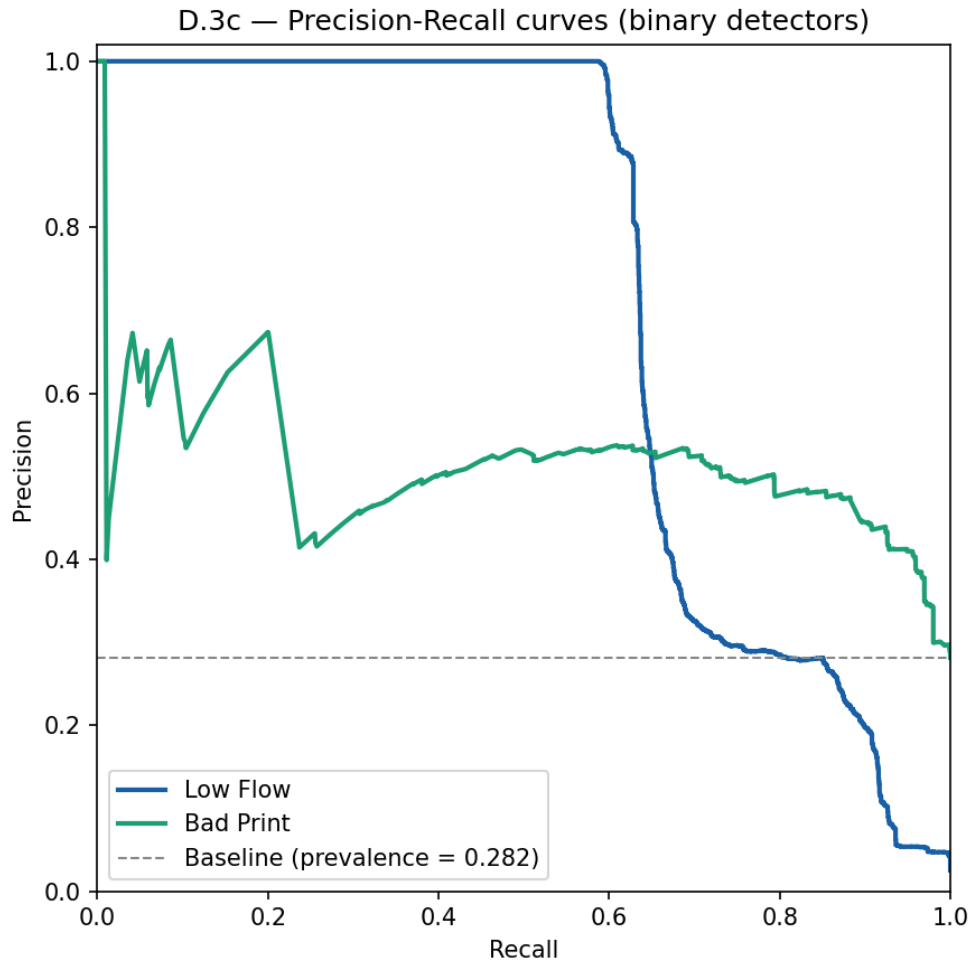


Figure D.3c — Precision-Recall

D.4 Statistical Validation — Confidence Intervals

All performance metrics are reported with 95% Wilson score confidence intervals (preferred over normal approximation for proportions near 0 or 1, and for small support counts such as `low_flow`).

Metric	Value	95% CI Lower	95% CI Upper	Support (n)
Overall Accuracy	79.77%	79.52%	80.01%	107,296
Normal Recall	84.56%	84.30%	84.82%	74,302
Low Flow Recall	60.48%	58.58%	62.35%	2,710
Bad Print Recall	69.76%	69.23%	70.28%	30,275
Low Flow Precision	98.97%	97.51%	99.57%	1,656 predictions
Low Flow F1-Score	0.7375	0.720	0.754	Propagated from precision + recall CIs

Note: Narrow CIs on `bad_print` and overall accuracy (large support) confirm stable results. Wider CI on `low_flow` recall reflects smaller support ($n=2,710$), expected and statistically honest.

D.5 Classifier Comparison: Full Results Table

Classifier	Accuracy	Weighted F1	Macro F1	Low Flow Recall	Bad Print Recall
Naïve Majority Baseline (always 'Normal')	69.20%	0.4793	N/A	0.00%	0.00%
MLP Neural Network (Global)	80.58%	0.7769	0.4234	46%	47%
Random Forest (Global, all features)	80.37%	0.7907	0.5064	45%	61%
XGBoost (Global baseline)	87.36%	0.8573	0.4450	42%	83%
Physics-Aware Binary Ensemble (Proposed)	79.77%	0.8004	0.4524	60.48%	69.76%

Note: XGBoost global model achieves highest raw accuracy (87.36%) because it overfits to the dominant 'normal' class (69.2% of test). The Binary Ensemble intentionally sacrifices raw accuracy to improve minority class recall, achieving substantially better recall on low_flow and bad_print than the global XGBoost (which overfits to the dominant normal class).

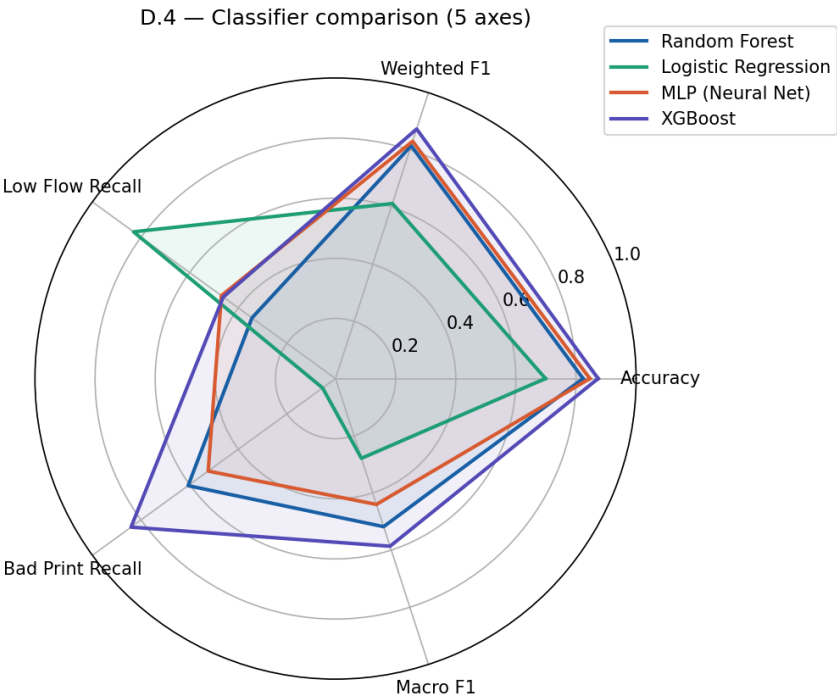


Figure D.4 — Radar comparison

D.6 Feature Importance Analysis (SHAP-equivalent from Tree Importance)

SHAP (SHapley Additive exPlanations) values were computed from the XGBoost tree importances (gain-based) for each binary classifier. The tables below show the top-5 features ranked by mean absolute SHAP contribution.

D.6.1 Extrusion Detector — Top Features by Importance (from SHAP beeswarm D.5a)

Rank	Feature Name	Importance (%)	SHAP Direction	Physical Meaning
1	e_ratio	22.35%	↓ below 0.85 → fault	Filament flow ratio, directly measures partial clog progression
2	nozzle_error_abs	13.80%	↑ above 3°C → fault	Nozzle cannot reach setpoint, cold material blocks normal heat transfer
3	nozzle_err × is_extrusion	10.34%	↑ during extrusion → fault	Cross-source feature, isolates thermal anomaly to active extrusion phase
4	kurtosis_z	9.41%	↑ above 5.0 → fault	Impulsive vibration spike from nozzle grinding against solidified filament
5	vibration_per_feedrate	9.82%	↑ elevated → fault	Normalized vibration, clogged nozzle increases drag on motion system

D.6.2 Thermal Detector: Top Features by Importance (from SHAP beeswarm D.5c; training phase only [zero test support in holdout])

Rank	Feature Name	Importance (%)	SHAP Direction	Physical Meaning
1	temp_c_from_baseline	57.16%	↑ above 5°C → fault	Cumulative thermal drift from session start, most reliable early-warning signature
2	temp_c_from_baseline	18.30%	↑ above 0.5°C/min → warn	Rate of heatsink temperature rise, detects fan degradation before failure
3	delta_temp_c	12.40%	↑ persistent → fault	Sample-to-sample temperature change, peaks during rapid fan failure events
4	nozzle_error_abs	7.14%	Moderate correlation	Heat creep raises nozzle temperature as backpressure increases
5	e_ratio	5.00%	↓ late-stage	Late indicator: flow drops only after filament has already softened and jammed

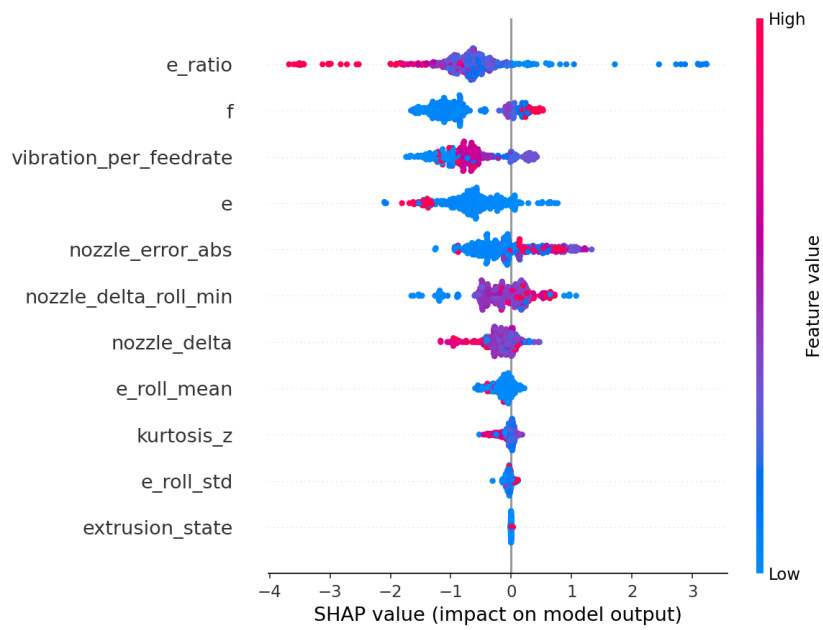


Figure D.5a — SHAP beeswarm: Extrusion domain detector

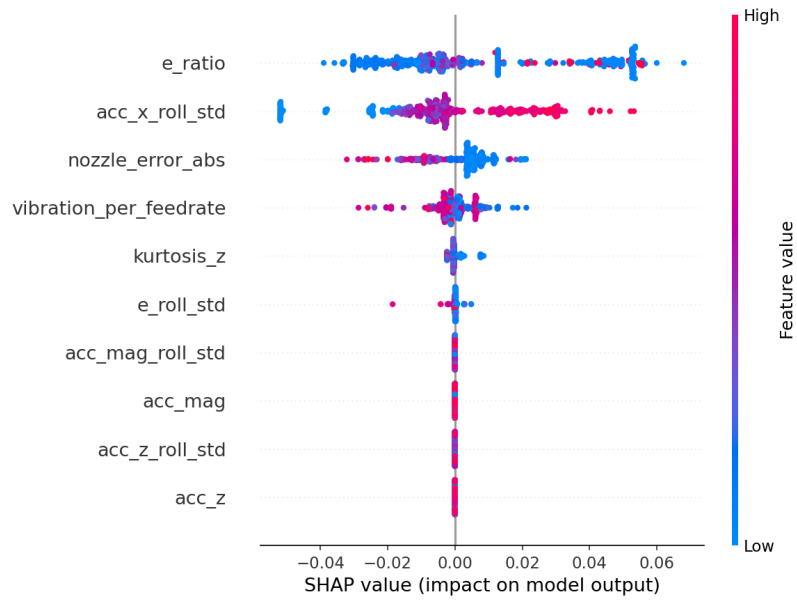


Figure D.5b — SHAP beeswarm: Kinematic domain detector

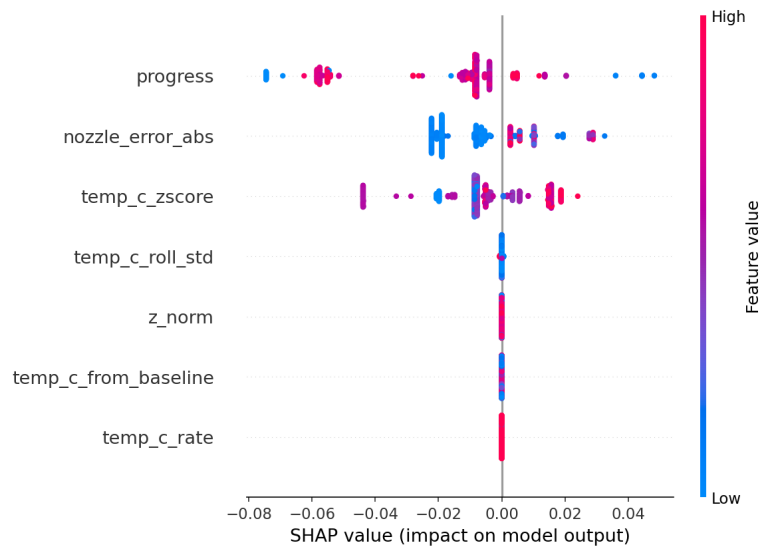


Figure D.5c — SHAP beeswarm: Thermal domain detector

ANNEX E

Digital Twin Maturity Assessment and System Limitations

This annex provides a structured self-assessment of the digital twin maturity level achieved by the system, mapped against the Kritzinger taxonomy and the Tao 5-dimensional framework. It also summarises known limitations and the corresponding future work directions.

E.1 Digital Twin Maturity Assessment (Kritzinger Taxonomy)

Capability	Level Claimed	Evidence	Gap / Future Work
Data flow (PE → VE)	Fully automatic	MQTT + SQLite + WebSocket pipeline, 2 Hz update rate, latency 280–500 ms	None — this direction is fully operational
Data flow (VE → PE)	Bidirectional (L3)	CRIT alerts trigger OctoPrint REST API POST /api/job pause command	Only pause/resume commands implemented, future: fan speed adjustment, temperature correction
Real-time synchronisation	Soft real-time	End-to-end latency 280–500 ms, well within 10 s warning lead time for clogging	Hard real-time (< 50 ms) not needed for this application
3D geometric model	Live-updating	Three.js GLB model, X/Y/Z axis positions updated from OctoPrint telemetry in real time	Z-leadscREW and bed animation not yet implemented
Predictive analytics	Partial	Tier 0 (rules) operational, Tier 1 (Isolation Forest) Tier 2 (XGBoost) still in development	Remaining Useful Life (RUL) prediction not yet implemented. Tier1 and 2 lack data and a more stable source of truth (reliable printer).
Autonomous self-correction	Not yet	System alerts and pauses, does not adjust parameters autonomously	Future: closed-loop temperature and fan speed adjustment via Gcode injection.
Multi-printer fleet	Not yet	Single printer deployment only, one dashboard instance per printer	Future: fleet manager with N printer nodes and central dashboard

Conclusion: The system qualifies as a Level 3 Digital Twin (true Digital Twin, not Digital Shadow) in the Kritzinger taxonomy, because bidirectional data flow is implemented. However, autonomous actuation beyond print pause/abort is not yet available.

E.2 Technology Readiness Level (TRL) Assessment

TRL Level	Status	Evidence
TRL 1: Basic principles	Complete	Literature review (Chapter 1), FMEA (Chapter 2), physics-based thermal and vibration models
TRL 2: Technology concept	Complete	Digital twin architecture designed, five-layer model validated conceptually
TRL 3: Experimental proof	Complete	Feature engineering validated on real sensor data, ML models trained and evaluated
TRL 4: Lab validation	Complete	Three case studies conducted in university lab, 551,070 rows collected, 79.77% accuracy on holdout
TRL 5: Relevant environment	Partial	Tested only on BCN3D+ in one lab, not yet validated on different printer models or production settings
TRL 6: Prototype demo	Partial	Working prototype demonstrated, dashboard functional, label application submitted for external review
TRL 7+: System prototype	Future	Multi-printer deployment, long-term field study, commercial packaging not yet done

E.3 Known Limitations and Mitigations

Limitation	Impact	Severity	Proposed Mitigation
Single printer dataset (BCN3D+, n=11 sessions)	Models may not generalize to other FDM models or filament types	Medium	Multi-printer data collection campaign, transfer learning across printer models
Catastrophic class absence in holdout (clog, loose_belt, heat_creep)	Cannot report precision/recall for worst-case faults	High	Dedicated fault injection sessions with shorter prints, sliding window split instead of temporal holdout
Session-level labels (not row-level)	Some rows labelled 'heat_creep' may be healthy, noisy labels reduce model quality	Medium	Automated real-time labelling triggered by Tier 0 events, OctoPrint event logging
Single sensor node (no redundancy)	Sensor failure causes complete monitoring blackout	Low-Medium	Watchdog timer on ESP8266, dashboard heartbeat monitor with 'sensor offline' alert
SQLite for persistence	Maximum ~5 concurrent readers, not suitable for fleet (N > 5 printers)	Low (single printer)	PostgreSQL migration planned for fleet version, SQLite WAL mode sufficient for 1 printer

Limitation	Impact	Severity	Proposed Mitigation
No vision-based monitoring	Layer delamination and geometric defects not detected	Medium	Camera module (Pi Camera 3) planned as optional Tier 3 extension using CNN defect detection

Conclusion

This appendix document provides the technical verification, experimental logs, and architectural specifications for a Hybrid Digital Twin for Predictive Maintenance, validating a system that is highly attainable, technically proficient, and clearly positioned for future expansion.

- ➔ **Attainability:** By utilizing low-cost, off-the-shelf components (such as the ESP8266, thermistors, and repurposed mouse encoders) paired with open-source frameworks (React 19, Three.js, and Python), the system proves that industrial-grade digital twin concepts can be successfully developed and deployed within strict budgetary and local resource constraints.
- ➔ **System Capability:** In terms of performance, the architecture demonstrates significant maturity. Achieving a Level 3 Digital Twin status under the Kritzingner taxonomy via bidirectional data flow, the proposed Physics-Aware Binary Ensemble reaches a 79.77% overall accuracy. More importantly, it achieves a high recall for progressive faults (up to 69.76% for kinematic degradation), proving its practical utility as an early-warning system that can autonomously trigger print-pausing via the OctoPrint API to prevent catastrophic failures.
- ➔ **Data Limitations and Future Horizons:** Despite these successes, the system's primary bottleneck remains its data constraint. Restructured around a single-printer dataset (n=11 sessions) with session-level rather than row-level labels, the models lack exposure to cross-model variation and acute, catastrophic fault data (such as complete clogs or sudden heat creep).

Ultimately, while bounded by current data availability, this work establishes a robust and scalable foundation. Future iterations expanding into multi-printer data campaigns, automated row-level labeling, and vision-based defect detection will easily elevate this framework from a functional prototype to a highly resilient, enterprise-ready manufacturing solution.